

Software Design Document (SDD)

Module: i2c_driver - STM32 I2C Driver

Item	Description
Version	1.0
Authors	Steven McClellan, Vania Leal, Leonardo Garcia, Melanie Picen
Source files	i2c_driver.c / i2c_driver.h
Main purpose	Initialize STM32 I2C peripherals and provide blocking I2C read/write transactions for 7-bit addressed devices.
Main public functions	i2c_init(), i2c_writeRegDevice(), i2c_writeDevice(), i2c_readRegDevice(), i2c_readDevice()

1. Introduction

1.1 Purpose

This document describes the design and architecture of the i2c_driver module. The module configures the STM32 I2C peripheral and provides a compact polling-based API for transmitting and receiving bytes from external I2C slave devices. It is intended to support higher-level modules such as color sensor drivers without requiring them to manipulate I2C registers directly.

1.2 Scope

The driver supports I2C1, I2C2, and I2C3 instances. It validates configuration arguments, enables the selected APB1 peripheral clock, resets and configures the I2C timing registers, and performs blocking write/read transactions with timeout and NACK handling. The implementation shown here is based on a 16 MHz APB1 clock and supports standard-mode and fast-mode speed calculations. GPIO pin configuration for SCL/SDA is outside the scope of this driver and must be completed before using the I2C peripheral.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
I2C	Inter-Integrated Circuit serial bus using SCL and SDA lines.
SCL	Serial Clock line driven by the master during communication.
SDA	Serial Data line shared by master and slave.
START	Bus condition generated by setting CR1.START to begin a transaction.
STOP	Bus condition generated by setting CR1.STOP to release the bus.
Repeated START	A second START without releasing the bus, used for register reads.
ACK / NACK	Acknowledge / Not Acknowledge response from receiver.
SB	Start Bit flag in SR1 indicating START condition generated.
ADDR	Address flag in SR1 indicating address phase completed.
TXE	Transmit data register empty flag.
RXNE	Receive data register not empty flag.
BTF	Byte transfer finished flag.
AF	Acknowledge failure flag used to detect NACK.
CCR	Clock Control Register used to program SCL timing.
TRISE	Maximum rise time register.
SDD	Software Design Document.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	i2c_init()	The system shall validate the I2C configuration, enable the selected APB1 peripheral clock, reset the peripheral, configure CR2/CCR/TRISE, and enable the I2C peripheral.
FR-2	i2c_writeRegDevice()	The system shall write one or more bytes to a device register using START, address write, register address, payload bytes, and STOP.
FR-3	i2c_writeDevice()	The system shall write a raw byte sequence to a 7-bit addressed I2C device without a register-address phase.
FR-4	i2c_readRegDevice()	The system shall read bytes from a device register using a write register-address phase followed by a repeated START and read phase.
FR-5	i2c_readDevice()	The system shall read a raw byte sequence from a 7-bit addressed I2C device without first writing a register address.
FR-6	All transaction functions	The system shall return timeout or NACK status when the bus or slave device does not respond as expected.

Non-functional requirements: no dynamic memory allocation, direct register access through STM32 register definitions, blocking/polling transaction model, timeout protection on wait loops, and simple status-code based error propagation.

2. System Architecture

2.1 High-Level Design

The I2C driver sits between application/device-driver modules and the STM32 I2C hardware registers. Higher-level code calls the public API. This driver performs register-level sequencing and returns status codes. External sensors or devices remain isolated behind the I2C transaction functions.

Layer	Files / Components	Description
Application / Device Driver Layer	main.c, tcs3472_driver.c, other sensor drivers	Calls the I2C driver to configure peripherals and exchange bytes with slave devices.
I2C Driver Layer	i2c_driver.c / i2c_driver.h	Implements initialization and blocking I2C read/write operations.
MCU Register Layer	I2C1/I2C2/I2C3, RCC APB1 registers	CR1, CR2, CCR, TRISE, SR1, SR2, and DR are accessed directly.
Hardware / Bus Layer	STM32 pins and external I2C devices	Physical SCL/SDA bus and 7-bit slave devices.

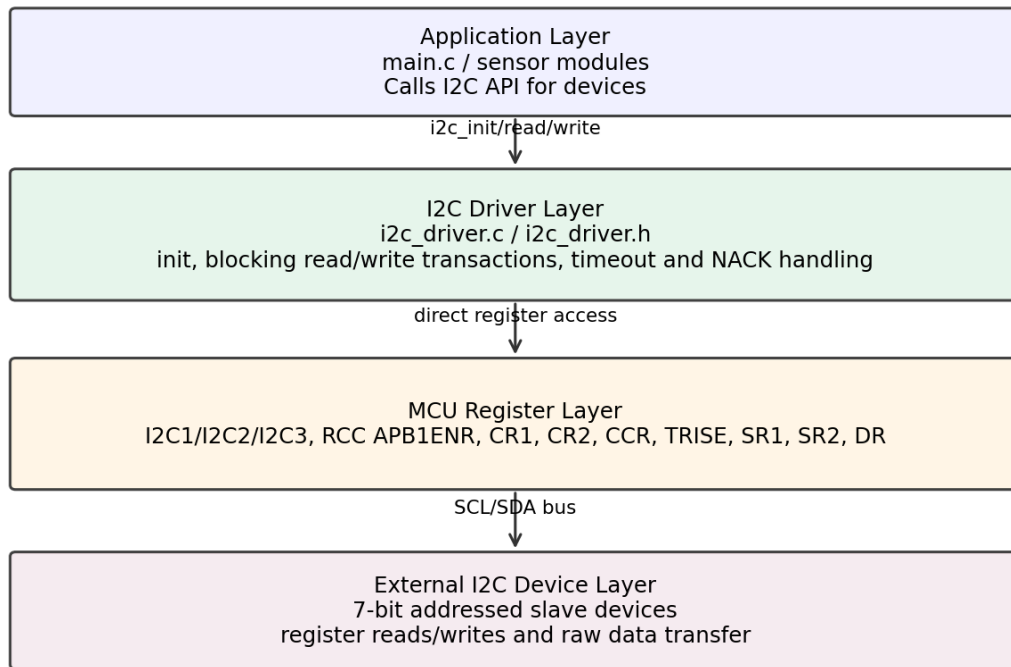


Figure 1. Class / Layer Diagram - I2C Driver

2.2 Module Interfaces

Interface	Description
Public API	<code>i2c_init()</code> , <code>i2c_writeRegDevice()</code> , <code>i2c_writeDevice()</code> , <code>i2c_readRegDevice()</code> , <code>i2c_readDevice()</code> declared in <code>i2c_driver.h</code> .
External dependencies	STM32 register definitions, <code>RCC_PERIPH()</code> , <code>I2C_RegDef_t</code> , <code>I2C1/I2C2/I2C3</code> base addresses, and I2C register/flag masks.
Configuration dependency	The application must configure the GPIO pins for SCL/SDA alternate function and open-drain behavior before I2C communication.

3. Detailed Design

3.1 Module Behavior

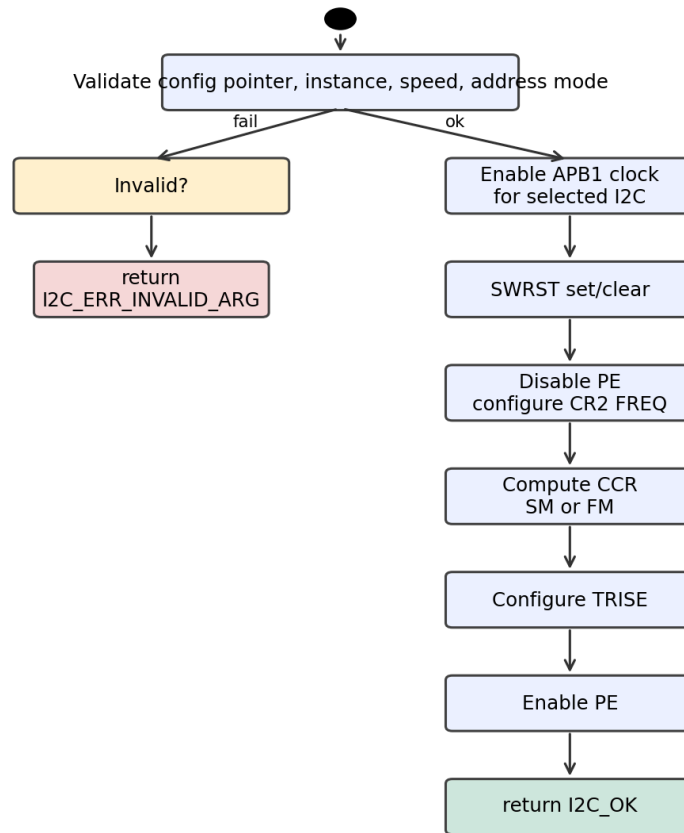


Figure 2. Activity Diagram - i2c_init

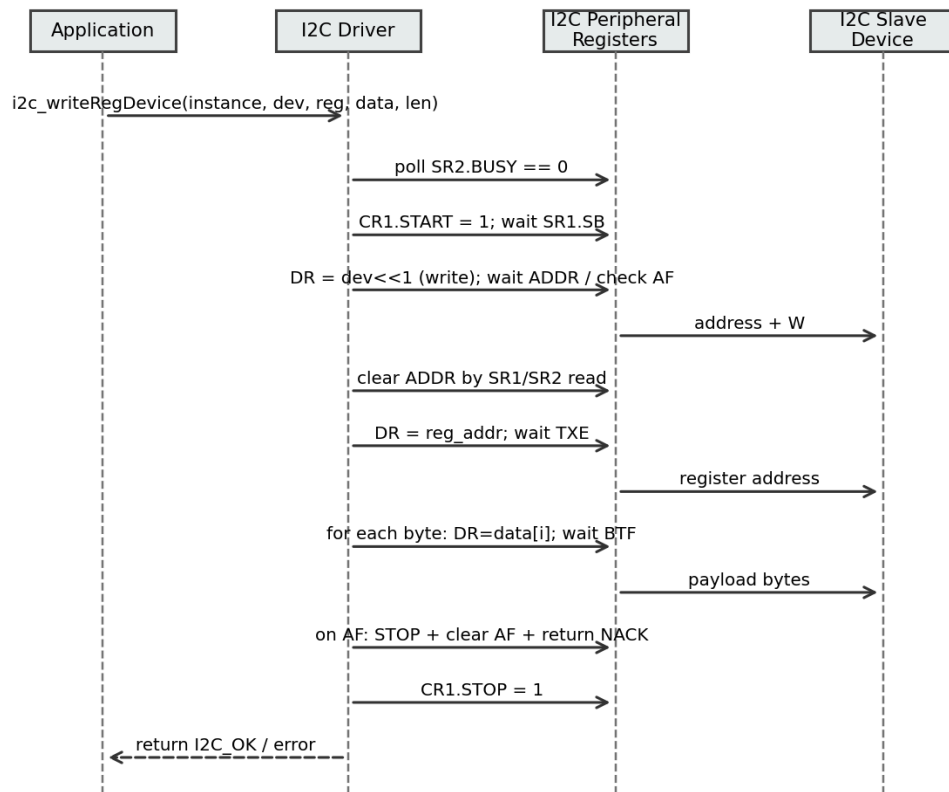


Figure 3. Sequence Diagram - Register Write Transaction

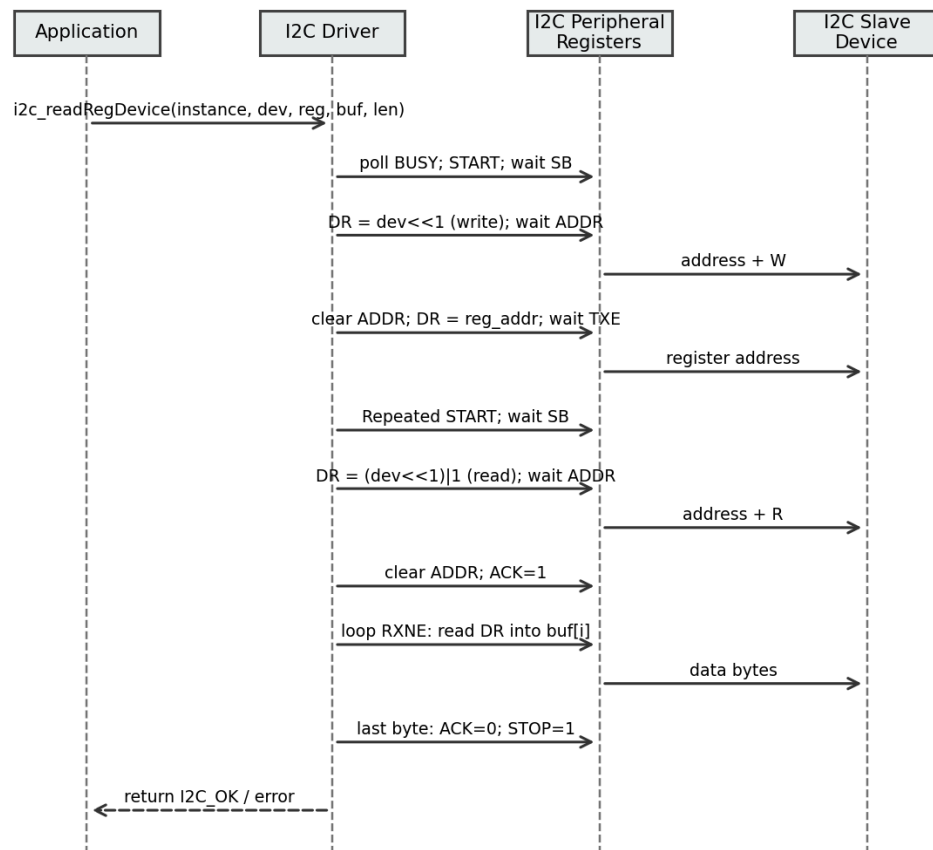


Figure 4. Sequence Diagram - Register Read Transaction

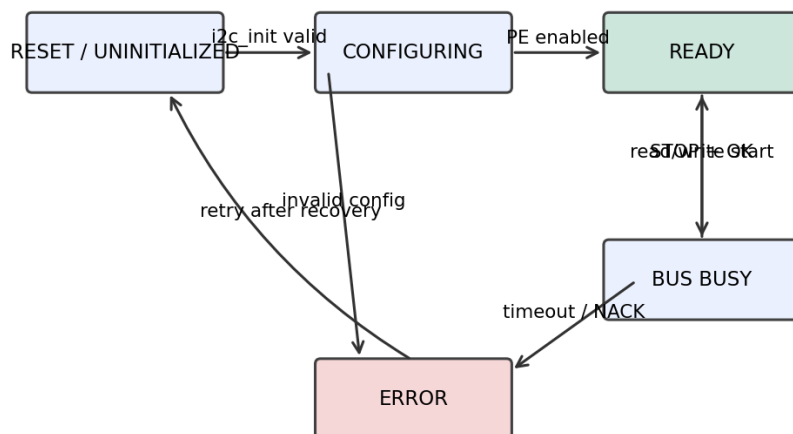


Figure 5. State Diagram - I2C Driver Lifecycle

3.2 Initialization Flow

`i2c_init()` first validates the configuration pointer, peripheral instance, requested clock speed, and address mode. It then enables the corresponding RCC APB1 clock bit for I2C1, I2C2, or I2C3. The peripheral is reset using the SWRST bit,

disabled for configuration, programmed with APB1 frequency, CCR timing, and TRISE timing, then enabled by setting CR1.PE.

The driver assumes `APB1_CLK_HZ = 16,000,000 Hz`. For standard mode, CCR is calculated as `APB1_CLK_HZ / (2 * clk_speed)`. For fast mode, CCR is calculated as `APB1_CLK_HZ / (3 * clk_speed)`.

3.3 Write Transaction Flow

The register write function waits for the bus to become free, generates START, waits for SB, sends the 7-bit device address shifted left for write mode, waits for ADDR, clears ADDR by reading SR1 and SR2, sends the register address, then writes each payload byte. Each wait loop is bounded by `I2C_TIMEOUT_MAX`. If AF is set during address or byte transfer, the driver generates STOP, clears AF, and returns `I2C_ERR_NACK`.

The raw write function follows the same general write sequence but omits the register-address phase. It waits for BTF for intermediate bytes and TXE for the final byte before generating STOP.

3.4 Read Transaction Flow

The register read function performs a two-part transaction. First, it writes the target register address using a write-address phase. Then it generates a repeated START, sends the same device address with the read bit set, enables ACK, and reads bytes as RXNE becomes set. Before the last byte, ACK is disabled and STOP is generated so that the bus is released after the final data byte.

The raw read function begins directly with the read address phase and follows the same ACK/RXNE loop used by the register read function.

4. Application Programming Interfaces

4.1 Data Types and Macros

Name	Type / Value	Notes / Definition
<code>APB1_CLK_HZ</code>	16000000U	Assumed APB1 peripheral clock frequency used for CR2, CCR, and TRISE calculations.
<code>i2c_config_t</code>	structure	Configuration structure expected to contain at least instance, clk_speed, and addr_mode.
<code>I2C_RegDef_t</code>	structure	Register layout for an STM32 I2C peripheral instance.
<code>I2C_TIMEOUT_MAX</code>	macro	Maximum polling count before returning <code>I2C_ERR_TIMEOUT</code> .
<code>I2C_SPEED_SM_HZ</code>	macro	Standard-mode speed threshold.
<code>I2C_SPEED_FM_HZ</code>	macro	Fast-mode maximum speed threshold used for validation.

4.2 Error Codes

Code	Notes / Definition
<code>I2C_OK</code>	Operation completed successfully.
<code>I2C_ERR_INVALID_ARG</code>	A pointer, peripheral instance, device address, clock speed, length, or address mode argument is invalid.
<code>I2C_ERR_TIMEOUT</code>	A hardware flag did not reach the expected state before <code>I2C_TIMEOUT_MAX</code> expired.
<code>I2C_ERR_NACK</code>	The slave did not acknowledge the address or a transmitted byte; detected through the AF flag.

4.3 Function Definitions

`i2c_init`

Field	Definition
Function Name	<code>i2c_init</code>
Syntax	<code>i2c_status_t i2c_init(const i2c_config_t *config)</code>

Parameters (in)	config: pointer to I2C configuration structure.
Parameters (inout)	None.
Parameters (out)	None.
Return value	i2c_status_t: I2C_OK or I2C_ERR_INVALID_ARG.
Description	Initializes and enables the selected I2C peripheral.
Constraints	GPIO alternate function/open-drain configuration must already be handled externally; APB1 clock is assumed to be 16 MHz.
Available via	i2c_driver.h

i2c_writeRegDevice

Field	Definition
Function Name	i2c_writeRegDevice
Syntax	i2c_status_t i2c_writeRegDevice(I2C_RegDef_t *instance, uint16_t dev_addr, uint8_t reg_addr, const uint8_t *data, size_t len)
Parameters (in)	instance, dev_addr, reg_addr, data, len.
Parameters (inout)	None.
Parameters (out)	I2C bus side effect: bytes are transmitted to the slave register.
Return value	i2c_status_t: I2C_OK, I2C_ERR_INVALID_ARG, I2C_ERR_TIMEOUT, or I2C_ERR_NACK.
Description	Writes len bytes to a specific register of a 7-bit I2C slave device.
Constraints	Rejects device addresses above 0x7F; blocking polling implementation.
Available via	i2c_driver.h

i2c_writeDevice

Field	Definition
Function Name	i2c_writeDevice
Syntax	i2c_status_t i2c_writeDevice(I2C_RegDef_t *instance, uint16_t dev_addr, const uint8_t *data, size_t len)
Parameters (in)	instance, dev_addr, data, len.
Parameters (inout)	None.
Parameters (out)	I2C bus side effect: bytes are transmitted to the slave.
Return value	i2c_status_t: I2C_OK, I2C_ERR_INVALID_ARG, I2C_ERR_TIMEOUT, or I2C_ERR_NACK.
Description	Writes len bytes directly to a 7-bit I2C slave device without a register address.
Constraints	Blocking polling implementation; caller must provide the exact payload expected by the device.
Available via	i2c_driver.h

i2c_readRegDevice

Field	Definition
Function Name	i2c_readRegDevice
Syntax	i2c_status_t i2c_readRegDevice(I2C_RegDef_t *instance, uint16_t dev_addr, uint8_t reg_addr, uint8_t *buf, size_t len)
Parameters (in)	instance, dev_addr, reg_addr, len.
Parameters (inout)	buf pointer is written during the read transaction.
Parameters (out)	buf receives len bytes from the target register sequence.
Return value	i2c_status_t: I2C_OK, I2C_ERR_INVALID_ARG, I2C_ERR_TIMEOUT, or I2C_ERR_NACK.
Description	Reads len bytes from a specific register of a 7-bit I2C slave device using repeated START.
Constraints	Blocking polling implementation; ACK is disabled for the last byte.
Available via	i2c_driver.h

i2c_readDevice

Field	Definition
Function Name	i2c_readDevice
Syntax	i2c_status_t i2c_readDevice(I2C_RegDef_t *instance, uint16_t dev_addr, uint8_t *buf, size_t len)
Parameters (in)	instance, dev_addr, len.

Parameters (inout)	buf pointer is written during the read transaction.
Parameters (out)	buf receives len bytes from the slave device.
Return value	i2c_status_t: I2C_OK, I2C_ERR_INVALID_ARG, I2C_ERR_TIMEOUT, or I2C_ERR_NACK.
Description	Reads len bytes directly from a 7-bit I2C slave device without first writing a register address.
Constraints	Blocking polling implementation; caller must know the device read behavior.
Available via	i2c_driver.h

5. Implementation Notes and Constraints

Topic	Note
Blocking execution	All transactions use polling loops. The CPU remains inside the driver until the transaction completes or a timeout/error occurs.
7-bit transaction limit	The initialization function validates 7-bit and 10-bit address modes, but transfer functions reject dev_addr values above 0x7F. Current transactions are therefore 7-bit only.
GPIO configuration external	The driver does not configure SCL/SDA pins. The application must set alternate function, open-drain output type, and pull-up behavior as required.
Timeout behavior	Timeouts prevent infinite waits. Some timeout exits return immediately without issuing STOP if the failure occurs before the transaction is fully active.
NACK handling	On AF flag detection, the driver generates STOP, clears AF, and returns I2C_ERR_NACK.
Clock assumption	Timing calculations assume APB1_CLK_HZ = 16 MHz. If the MCU clock tree changes, this macro must be updated.
Interrupt/DMA support	The current implementation does not use interrupts or DMA.

6. Future Enhancements

- Add explicit 10-bit addressing support or restrict i2c_config_t validation to 7-bit mode only.
- Add bus recovery logic for stuck SDA/SCL conditions after timeout.
- Move repeated validation and instance selection into private helper functions to reduce duplicated code.
- Add non-blocking or interrupt-driven transaction support for longer transfers.
- Improve single-byte and multi-byte read handling according to STM32 reference-manual timing requirements.
- Allow APB1 clock frequency to be passed through configuration instead of fixed APB1_CLK_HZ.
- Add unit tests or hardware-in-loop tests for valid transaction flow, NACK, timeout, invalid argument, and boundary lengths.